L39 — B-Trees: Structure and Operations

Unit: 3 | **Chapter:** Trees | **BT Level:** BT6 | **CO:** CO2, CO4

Learning Objectives

- Define B-tree and its structural properties
- Insert and delete keys in a B-tree with splitting and merging
- Explain why B-trees are preferred for disk-based storage
- Distinguish B-trees from B+ trees and binary trees

---

1. Motivation: Disk-Based Data Structures

For in-memory data, AVL/Red-Black trees with $O(\log_2 n)$ performance are excellent.

For **disk-based storage** (databases, file systems):

- Disk reads are **1000× slower** than memory reads
- Each "disk access" reads a fixed-size **page** (e.g., 4KB)
- Goal: minimize the number of disk accesses = minimize tree height

B-tree: A generalization of BST where each node can contain **many keys** and have **many children**, reducing tree height drastically.

---

2. B-Tree Definition (Order t)

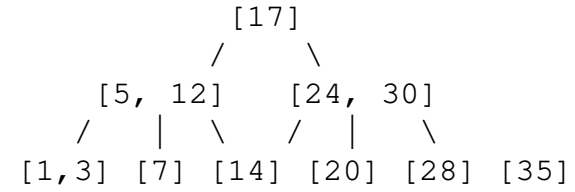
A B-tree of minimum degree **t** ($t \geq 2$) satisfies:

1. Every node has at most **2t - 1 keys** (max 2t children)
2. Every non-root node has at least **t - 1 keys** (min t children)
3. All leaves have the same depth (perfectly balanced)
4. Keys in each node are **sorted**
5. For any key k[i] in a node, all keys in child[i] $<$ k[i] $<$ all keys in child[i+1]

B-tree of order t=2 (2-3-4 tree): Nodes have 1–3 keys, 2–4 children.

---

3. B-Tree Structure (t=2)



- Height h ≈ log_t(n) → very shallow
- Each node is a disk page → few disk accesses per operation

4. Node Representation

```
class BTreeNode:
    def __init__(self, leaf=False):
        self.keys = []          # List of keys (sorted)
        self.children = []      # List of child nodes
        self.leaf = leaf        # True if this is a leaf node

class BTree:
    def __init__(self, t):
        self.t = t               # Minimum degree
        self.root = BTreeNode(leaf=True)
```

---

5. Search

```
def search(self, k, node=None):
    """Search for key k. Returns (node, index) or None."""
    if node is None:
        node = self.root

    i = 0
    while i < len(node.keys) and k > node.keys[i]:
        i += 1

    if i < len(node.keys) and k == node.keys[i]:
        return (node, i)   # Found!

    if node.leaf:
        return None        # Not found

    return self.search(k, node.children[i])   # Recurse into child
```

Trace: search(14) in tree above:

1. Root [17]: 14 < 17 → go to child[0] = [5, 12]
2. [5, 12]: 14 > 12 → go to child[2] = [14]
3. [14]: 14 = = 14 → **Found!**

---

6. Insertion

Key idea: Insert always at a leaf. If a node becomes **overfull** (2t keys), **split** it by pushing the median key up to the parent.

6.1 Split a Full Child

```
def _split_child(self, parent, i):
    """Split the i-th child of parent (which is full with 2t-1 keys)."""
    t = self.t
```

```
    full_child = parent.children[i]
    new_child = BTreeNode(leaf=full_child.leaf)

    # Median key
    mid = t - 1

    # Move median to parent
    parent.keys.insert(i, full_child.keys[mid])
    parent.children.insert(i + 1, new_child)

    # Split keys and children
    new_child.keys = full_child.keys[mid + 1:]
    full_child.keys = full_child.keys[:mid]

    if not full_child.leaf:
        new_child.children = full_child.children[t:]
        full_child.children = full_child.children[:t]
```

6.2 Insert Key

```
def insert(self, k):
    root = self.root

    if len(root.keys) == 2 * self.t - 1:   # Root is full → split
        new_root = BTreeNode(leaf=False)
        new_root.children.append(self.root)
        self._split_child(new_root, 0)
        self.root = new_root

    self._insert_non_full(self.root, k)

def _insert_non_full(self, node, k):
    i = len(node.keys) - 1

    if node.leaf:
        # Insert key in sorted position
        node.keys.append(None)
        while i >= 0 and k < node.keys[i]:
            node.keys[i + 1] = node.keys[i]
            i -= 1
        node.keys[i + 1] = k
    else:
        # Find appropriate child
        while i >= 0 and k < node.keys[i]:
            i -= 1
        i += 1

        if len(node.children[i].keys) == 2 * self.t - 1:
            self._split_child(node, i)
            if k > node.keys[i]:
                i += 1

        self._insert_non_full(node.children[i], k)
```

7. Insertion Trace (t = 2)

Initial B-tree: Root = [10, 20, 30, 40] (full, 2t-1 = 3 means max 3 keys; here t = 2)

Inserting 25:

1. Root [10, 20, 30] is not full (t = 2, max = 3 keys) → go to appropriate child
2. 25 is between 20 and 30 → go to middle child
3. Insert 25 in leaf → leaf now has [25] → Done

Inserting 35 (causes split): If leaf [30, 35, 40] becomes full (3 keys = 2t-1):

- Median = 35
- Push 35 up to parent
- Split: left child [30], right child [40]

---

8. Deletion (Overview)

Deletion is more complex — key may be in internal node or leaf.

Three cases:

1. **Key in leaf:** Delete directly (if node has ≥ t keys after deletion)
2. **Key in internal node:** Replace with in-order predecessor/successor; then delete from leaf
3. **Node underflows (< t-1 keys):**
    - **Borrow** from a sibling (rotation through parent)
    - **Merge** with sibling + parent key (reduces parent's keys by 1)

---

9. B-Tree vs B+ Tree

Feature	B-Tree	B+ Tree
Data stored	In all nodes	Only in leaf nodes
Leaf linking	No	Leaves linked (for range queries)
Space efficiency	Lower	Higher (internal = index only)
Range queries	Slower	Fast (scan linked leaves)
Use in databases	Less common	Standard (MySQL InnoDB, PostgreSQL)

---

10. Complexity Analysis

Operation	Time
Search	O(t · log_t n)
Insert	O(t · log_t n)
Delete	O(t · log_t n)

Height	O(log_t n)

For $t = 500$ (typical disk page size), a B-tree with 1 billion keys has height $\approx \log_{500}(10^9) \approx 3$ levels → only **3 disk reads** to find any key!

Summary

- B-tree of degree t: each node has t-1 to 2t-1 keys; perfectly balanced; all leaves at same depth.
- **Search, insert, delete:** O(t · log_t n) — very shallow, minimizes disk I/O.
- **Insert:** Split full nodes proactively on the way down (or on overflow).
- **Delete:** Merge or borrow from siblings to maintain minimum key count.
- B+ trees (standard in databases) store data only in leaves, linked for range scans.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 18 (MIT Press, 2022)
- Ramakrishnan & Gehrke, *Database Management Systems*, Ch. 9 (McGraw-Hill, 2002)